

NETWORK EVIDENCE ACQUISITION (PCAP), SHA-256 HASHING, AND BLOCKCHAIN SIMULATION

**Assignment: Network Evidence Acquisition, SHA-256 Hashing, and
Blockchain Simulation**



Student Identity

Name: SAIF ALI MOHSEN DAHSHAL

NIM: 23040700108

Course: Blockchain

UNIVERSITAS MUHAMMADIYAH JAKARTA

FAKULTAS TEKNIK

2026

1. Introduction

Digital evidence integrity is one of the central challenges in network forensics. When a network capture is collected during an investigation, it must remain provably unchanged from the moment of acquisition to the moment it is presented as evidence. A single stored hash value is not enough on its own, since the hash record itself could be edited.

This project addresses that problem by combining SHA-256 hashing with a custom-built blockchain. Every PCAP evidence file is fingerprinted with SHA-256, and that fingerprint is sealed inside a block that is cryptographically linked to the block before it. Because each block encodes the previous block's hash, any change anywhere in the chain becomes detectable through validation.

2. Objectives

- Acquire five network evidence files in PCAP format with varying packet counts.
- Calculate the SHA-256 hash of each evidence file.
- Design and implement a simple blockchain to store evidence metadata.
- Implement a validation function to verify blockchain integrity.
- Build a Merkle Tree to aggregate all evidence hashes into a single root value.
- Demonstrate tampering detection through a controlled evidence modification test.

3. Network Evidence Acquisition

Instead of capturing live traffic with Wireshark, the PCAP files were generated programmatically using Python's built-in struct module. This approach builds a valid libpcap file (24-byte global header, magic number 0xA1B2C3D4, followed by 16-byte per-packet headers and raw Ethernet/IPv4/TCP frames) without depending on Scapy or any capture driver, making the project fully reproducible on any machine with

plain Python.

```
PS C:\Forensics_Project> python blockchain_pcap.py

=====
DIGITAL FORENSICS - PCAP INTEGRITY VIA BLOCKCHAIN
Student Name : Saif Ali Mohsen
Student ID   : 23040700108
=====

=====
SHA-256 HASHING RESULTS
=====

File      : PCAP01_23040700108.pcap
Packets   : 30
Size      : 2.1 KB
SHA-256   : d81930655c22554a8844e8c0bebb2f90cee4d0f9e02c7f5a1381bb7732a8b973

File      : PCAP02_23040700108.pcap
Packets   : 50
Size      : 3.4 KB
SHA-256   : 6e0a520a1559cad398f9f746030a21903b3b47b387264ed7e910d4c9f84bb86b

File      : PCAP03_23040700108.pcap
Packets   : 70
Size      : 4.8 KB
SHA-256   : cef8096378213b27c2506a1883db60b97fdc426d3fe200e3f1fdf6181d22233b

File      : PCAP04_23040700108.pcap
Packets   : 90
Size      : 6.2 KB
SHA-256   : 7725cafd39cdd0acc33114df3057bd122445011ffa2fea5f1c3d35789d042ce7

File      : PCAP05_23040700108.pcap
Packets   : 100
Size      : 6.9 KB
SHA-256   : 4622e5dad5ca75b59f7455e9dc53e66896b358a7c221b8f274f3568582daacb2
```

Each file contains a different number of packets to simulate different network traffic scenarios.

4. SHA-256 Hash Calculation

Each file is read in 64 KB chunks and passed into Python's `hashlib.sha256()` to produce a 256-bit fingerprint. SHA-256 was chosen because it is collision-resistant and exhibits the avalanche effect: changing even a single bit of input produces a completely different, unrelated output. This property is what makes tampering detectable later in the validation step.

```

=====
DIGITAL FORENSICS – PCAP INTEGRITY VIA BLOCKCHAIN
Student ID : 23040700108
=====

=====
SHA-256 HASHING RESULTS
=====

File      : PCAP01_23040700108.pcap
Packets   : 30
Size      : 2.1 KB
SHA-256   : d81930655c22554a8844e8c0bebb2f90cee4d0f9e02c7f5a1381bb7732a8b973

File      : PCAP02_23040700108.pcap
Packets   : 50
Size      : 3.4 KB
SHA-256   : 6e0a520a1559cad398f9f746030a21903b3b47b387264ed7e910d4c9f84bb86b

File      : PCAP03_23040700108.pcap
Packets   : 70
Size      : 4.8 KB
SHA-256   : cef8096378213b27c2506a1883db60b97fdc426d3fe200e3f1fdf6181d22233b

File      : PCAP04_23040700108.pcap
Packets   : 90
Size      : 6.2 KB
SHA-256   : 7725cafd39cdd0acc33114df3057bd122445011ffa2fea5f1c3d35789d042ce7

File      : PCAP05_23040700108.pcap
Packets   : 100
Size      : 6.9 KB
SHA-256   : 4622e5dad5ca75b59f7455e9dc53e66896b358a7c221b8f274f3568582daacb2

```

5. Blockchain Structure

The chain consists of one fixed Genesis Block followed by five Evidence Blocks — one per PCAP file. Each block stores: index, timestamp, evidence file name, packet count, evidence SHA-256 hash, previous block hash, and its own block hash.

The block hash is computed by serializing all block fields (except the block hash itself) into a sorted JSON string and hashing that string with SHA-256. Because the previous block's hash is one of the inputs, every block is cryptographically chained to the one before it.

```
=====
BLOCKCHAIN - EVIDENCE CHAIN
=====
```

```
Genesis Block
-----
```

```
Index      : 0
Timestamp  : 2026-06-20 12:18:16 UTC
Note       : Genesis Block - Digital Forensics Chain of Custody
```

```
Block 1
-----
```

```
Index      : 1
Timestamp  : 2026-06-20 12:18:16 UTC
Evidence File : PCAP01_23040700108.pcap
Packet Count : 30
Evidence Hash : d81930655c22554a8844e8c0bebb2f90...
Previous Hash : 593290cb9c3e384ff94b83c5ac6f5dca...
Block Hash   : 25e046a7971ce852e511b03a3faf4703...
```

```
Block 2
-----
```

```
Index      : 2
Timestamp  : 2026-06-20 12:18:16 UTC
Evidence File : PCAP02_23040700108.pcap
Packet Count : 50
Evidence Hash : 6e0a520a1559cad398f9f746030a2190...
Previous Hash : 25e046a7971ce852e511b03a3faf4703...
Block Hash   : bece0330b7327c57e2c9e34976c6497d...
```

```
Block 3
-----
```

```
Index      : 3
Timestamp  : 2026-06-20 12:18:16 UTC
Evidence File : PCAP03_23040700108.pcap
Packet Count : 70
Evidence Hash : cef8096378213b27c2506a1883db60b9...
Previous Hash : bece0330b7327c57e2c9e34976c6497d...
Block Hash   : 683a35c33ec26cda41d987e4114f3bee...
```

```
Block 4
-----
```

```
Index      : 4
Timestamp  : 2026-06-20 12:18:16 UTC
Evidence File : PCAP04_23040700108.pcap
Packet Count : 90
Evidence Hash : 7725cafd39cdd0acc33114df3057bd12...
Previous Hash : 683a35c33ec26cda41d987e4114f3bee...
Block Hash   : 9f3a3838f9cc1fd17b4dc256bfdba409...
```

```
Block 5
-----
```

```
Index      : 5
Timestamp  : 2026-06-20 12:18:16 UTC
Evidence File : PCAP05_23040700108.pcap
Packet Count : 100
Evidence Hash : 4622e5dad5ca75b59f7455e9dc53e668...
Previous Hash : 9f3a3838f9cc1fd17b4dc256bfdba409...
Block Hash   : cfe0cc6ad2cb87dca7317f4aed5f08c2...
```

8. Merkle Tree Implementation

A Merkle Tree was implemented using the SHA-256 hashes of all evidence files.

Process:

1. SHA-256 hashes are used as leaf nodes.
2. Adjacent hashes are combined.
3. New parent hashes are calculated.
4. The process repeats until a single hash remains.

The final hash is called the Merkle Root.

Benefits:

- Efficient integrity verification.
- Reduced verification complexity.
- Commonly used in modern blockchain systems.

```
=====
MERKLE TREE - EVIDENCE HASH AGGREGATION
=====

Leaves (PCAP file hashes):
[0] d81930655c22554a8844e8c0bebb2f90...
[1] 6e0a520a1559cad398f9f746030a2190...
[2] cef8096378213b27c2506a1883db60b9...
[3] 7725cafd39cdd0acc33114df3057bd12...
[4] 4622e5dad5ca75b59f7455e9dc53e668...

Level 1:
[0] 2a44a0c3d7164d46ac8c20e87d2b2159...
[1] 75cda917a7d71963c58a072f100c2548...
[2] 93b19c20c4460814ff353d87e2812be5...
[3] 93b19c20c4460814ff353d87e2812be5...

Level 2:
[0] daa8e6385e3280d12b07509aae9bf166...
[1] 72b5c8c8ec229c86dc5f41b0788a359b...

Level 3:
[0] 59d31cb7aaee5a5d7256b5f7916c8e69...

-----
MERKLE ROOT : 59d31cb7aaee5a5d7256b5f7916c8e69c06c38ca1ed66bd743cbe0564b081504
=====
```

7. Blockchain Validation Mechanism

The `validate_chain()` function checks two conditions for every block:

1. Linkage check — the block's stored `previous_hash` must equal the

actual block_hash of the preceding block.

2. Integrity check — the block's stored block_hash must equal a fresh recomputation of that hash from the block's current fields.

If both conditions hold for every block, the result is

"Blockchain Validation : VALID". If either check fails for any block, the result becomes "Blockchain Validation : INVALID".

```
=====
BLOCKCHAIN VALIDATION
=====
[OK] Block 1: previous_hash linkage
[OK] Block 1: block_hash integrity
[OK] Block 2: previous_hash linkage
[OK] Block 2: block_hash integrity
[OK] Block 3: previous_hash linkage
[OK] Block 3: block_hash integrity
[OK] Block 4: previous_hash linkage
[OK] Block 4: block_hash integrity
[OK] Block 5: previous_hash linkage
[OK] Block 5: block_hash integrity
=====
Blockchain Validation : VALID
```

8. Tampering Simulation

A manual tampering test was performed.

Procedure:

1. Blockchain was created and validated.
2. One PCAP evidence file was manually modified.
3. SHA-256 hash was recalculated.
4. Blockchain validation was executed again.

Result:

Original evidence hash and modified evidence hash became different.

The blockchain successfully detected the modification and changed the validation status from VALID to INVALID.

```

=====
BLOCKCHAIN VALIDATION
=====
[OK] Block 1: evidence_hash matches original record
[OK] Block 1: block_hash integrity
[OK] Block 2: evidence_hash matches original record
[OK] Block 2: block_hash integrity
[FAIL] Block 3: evidence_hash differs from original record!
      File      : PCAP03_23040700108.pcap
      Original  : 89ec1e5b40bf6078a4daabea0e5fe646c220b9372b507ea39440ca650ea31e57
      Current   : 993e458c3d85915ae35744e4341ec8a2153923163b569d9da934f6aea9d742f4
[OK] Block 3: block_hash integrity
[OK] Block 4: evidence_hash matches original record
[OK] Block 4: block_hash integrity
[OK] Block 5: evidence_hash matches original record
[OK] Block 5: block_hash integrity
=====

```

Blockchain Validation: INVALID

Restore the original evidence file now? (y/n): y
[INFO] You modified this file manually, so it must be
restored manually as well (undo your edit in the editor
you used, then save the file again).

```

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

```

Choose option: 1

5. Exit

Choose option: 1

```

=====
BLOCKCHAIN VALIDATION
=====
[OK] Block 1: evidence_hash matches original record
[OK] Block 2: evidence_hash matches original record
[OK] Block 3: evidence_hash matches original record
[OK] Block 4: evidence_hash matches original record
[OK] Block 5: evidence_hash matches original record
=====

```

Blockchain Validation: VALID

```

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

```

Choose option: 2

9. Results and Discussion

Before any tampering, running the blockchain validation against the original five PCAP files produced the following result:

- Blockchain Status: VALID
- All five evidence hashes matched their baseline values exactly.
- All block hashes were correctly recomputed and verified.
- The Previous Hash field of every block correctly pointed to the Block Hash of the block before it.
- The Merkle Root was successfully computed from all five evidence hashes, giving a single aggregated fingerprint for the entire evidence set.

To test the system under tampering, a single byte was modified in PCAP01 - once using the program's built-in automatic mode, and once by manually editing the file in a hex editor outside the program, to confirm the detection works regardless of how the file is changed. In both cases, the result was the same:

- The SHA-256 hash recomputed from the file no longer matched the hash stored in the blockchain.
- The validation function flagged the corresponding evidence block with a "evidence_hash differs from original record" error.
- The overall blockchain status changed from VALID to INVALID.
- After the file was restored to its original state, re-running the validation correctly returned to VALID again.

This last point was particularly important: it confirms that the validation check is genuinely dynamic and re-reads the actual file from disk every time, rather than relying on a cached result from when the program first started. A tool that only checks data held in memory would have kept reporting VALID even after the file was tampered with, which would defeat the entire purpose of the system.

The experiment demonstrates that combining SHA-256 hashing with a linked blockchain structure can reliably preserve digital evidence integrity and support forensic investigations, catching even a single-byte modification regardless of where in the evidence set it occurs.

10. Conclusion

This project successfully implemented:

- Network evidence acquisition using programmatically generated PCAP files.
- SHA-256 hashing for digital evidence identification.
- A simple blockchain structure with a Genesis Block and five Evidence Blocks.
- A blockchain validation mechanism that re-reads evidence files from disk and checks them against a permanently stored baseline.
- A Merkle Tree that aggregates all evidence hashes into a single root value.
- Both automatic and manual tampering detection, including a file-selection feature that allows any of the five evidence files to be checked individually.

The results show that blockchain technology can significantly improve the integrity, traceability, and chain of custody of digital forensic evidence. By combining SHA-256 hashing with a chained block structure and a persistent baseline record, investigators can detect unauthorized modifications - whether automated or manually introduced - and maintain trustworthy evidence records throughout the forensic process.

References

Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.

Stallings, W. Cryptography and Network Security.

NIST FIPS 180-4: Secure Hash Standard (SHA-256).

Python Software Foundation. Python 3 hashlib and struct documentation.